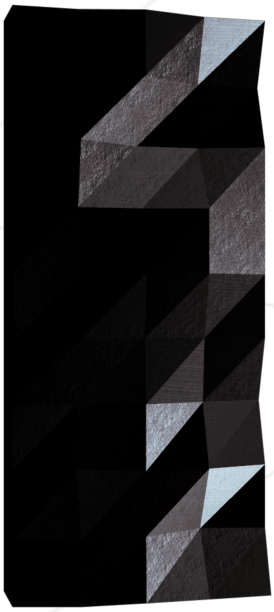

Minitalk

Summary:
The purpose of this project is to code a small data exchange program using UNIX signals.

Version: 5.0

Contents

I	**Foreword**	**2**
II	**Common Instructions**	**3**
III	**AI Instructions**	**5**
IV	**Project instructions**	**7**
V	**Mandatory Part**	**8**
VI	**Readme Requirements**	**9**
VII	**Bonus part**	**10**
VIII	**Submission and peer-evaluation**	**11**

Chapter I

Foreword

The cis-3-Hexen-1-ol, also known as (Z)-3-hexen-1-ol and leaf alcohol, is a colorless oily liquid with an intense grassy-green odor of freshly cut green grass and leaves.

It is produced in small amounts by most plants, and it acts as an attractant to many predatory insects. Cis-3-Hexen-1-ol is a very important aroma compound that is used in fruit and vegetable flavors and in perfumes.

The annual production is approximately 30 metric tons.

Chapter II

Common Instructions

- Your project must be written in C.
- Your project must be written in accordance with the Norm. If you have bonus files/functions, they are included in the norm check, and you will receive a `0` if there is a norm error.
- Your functions should not quit unexpectedly (segmentation fault, bus error, double free, etc.) except for undefined behavior. If this occurs, your project will be considered non-functional and will receive a `0` during the evaluation.
- All heap-allocated memory must be properly freed when necessary. Memory leaks will not be tolerated.
- If the subject requires it, you must submit a `Makefile` that compiles your source files to the required output with the flags `-Wall`, `-Wextra`, and `-Werror`, using `cc`. Additionally, your `Makefile` must not perform unnecessary relinking.
- Your `Makefile` must contain at least the rules `$(NAME)`, `all`, `clean`, `fclean` and `re`.
- To submit bonuses for your project, you must include a `bonus` rule in your `Makefile`, which will add all the various headers, libraries, or functions that are not allowed in the main part of the project. Bonuses must be placed in `_bonus.{c/h}` files, unless the subject specifies otherwise. The evaluation of mandatory and bonus parts is conducted separately.
- If your project allows you to use your `libft`, you must copy its sources and its associated `Makefile` into a `libft` folder. Your project's `Makefile` must compile the library by using its `Makefile`, then compile the project.
- We encourage you to create test programs for your project, even though this work **does not need to be submitted and will not be graded**. It will give you an opportunity to easily test your work and your peers' work. You will find these tests especially useful during your defence. Indeed, during defence, you are free to use your tests and/or the tests of the peer you are evaluating.
- Submit your work to the assigned Git repository. Only the work in the Git repository will be graded. If Deepthought is assigned to grade your work, it will occur

after your peer-evaluations. If an error happens in any section of your work during Deepthought's grading, the evaluation will stop.

Chapter III

AI Instructions

Context

During your learning journey, AI can assist with many different tasks. Take the time to explore the various capabilities of AI tools and how they can support your work. However, always approach them with caution and critically assess the results. Whether it's code, documentation, ideas, or technical explanations, you can never be completely sure that your question was well-formed or that the generated content is accurate. Your peers are a valuable resource to help you avoid mistakes and blind spots.

Main message

- Use AI to reduce repetitive or tedious tasks.
- Develop prompting skills — both coding and non-coding — that will benefit your future career.
- Learn how AI systems work to better anticipate and avoid common risks, biases, and ethical issues.
- Continue building both technical and power skills by working with your peers.
- Only use AI-generated content that you fully understand and can take responsibility for.

Learner rules:

- You should take the time to explore AI tools and understand how they work, so you can use them ethically and reduce potential biases.
- You should reflect on your problem before prompting — this helps you write clearer, more detailed, and more relevant prompts using accurate vocabulary.
- You should develop the habit of systematically checking, reviewing, questioning, and testing anything generated by AI.
- You should always seek peer review — don't rely solely on your own validation.

Phase outcomes:

- Develop both general-purpose and domain-specific prompting skills.
- Boost your productivity with effective use of AI tools.
- Continue strengthening computational thinking, problem-solving, adaptability, and collaboration.

Comments and examples:

- You'll regularly encounter situations — exams, evaluations, and more — where you must demonstrate real understanding. Be prepared, keep building both your technical and interpersonal skills.
- Explaining your reasoning and debating with peers often reveals gaps in your understanding. Make peer learning a priority.
- AI tools often lack your specific context and tend to provide generic responses. Your peers, who share your environment, can offer more relevant and accurate insights.
- Where AI tends to generate the most likely answer, your peers can provide alternative perspectives and valuable nuance. Rely on them as a quality checkpoint.

✓ Good practice:

> I ask AI: "How do I test a sorting function?" It gives me a few ideas. I try them out and review the results with a peer. We refine the approach together.

✗ Bad practice:

> I ask AI to write a whole function, copy-paste it into my project. During peer-evaluation, I can't explain what it does or why. I lose credibility — and I fail my project.

✓ Good practice:

> I use AI to help design a parser. Then I walk through the logic with a peer. We catch two bugs and rewrite it together — better, cleaner, and fully understood.

✗ Bad practice:

> I let Copilot generate my code for a key part of my project. It compiles, but I can't explain how it handles pipes. During the evaluation, I fail to justify and I fail my project.

Chapter IV

Project instructions

- Name your executable files `client` and `server`.
- You must submit a `Makefile` that compiles your source files. It must not perform unnecessary relinking.
- You are allowed to use your `libft`.
- You have to handle errors thoroughly. Under no circumstances should your program quit unexpectedly (segmentation fault, bus error, double free, and so forth).
- Your program must be free of **memory leaks**.
- You are allowed to use **one global variable per program** (one for the client and one for the server), but its usage must be justified.
- To complete the mandatory part, you are allowed to use the following functions:
  - `write`
  - `ft_printf` or any equivalent YOU coded
  - `signal`
  - `sigemptyset`
  - `sigaddset`
  - `sigaction`
  - `kill`
  - `getpid`
  - `malloc`
  - `free`
  - `pause`
  - `sleep`
  - `usleep`
  - `exit`

Chapter V

Mandatory Part

You must create a communication program in the form of a **client** and a **server**.

- The server must be started first. Upon launch, it must print its `PID`.
- The client takes two parameters:
  - The server `PID`.
  - The string to send.

these are the two args the client exec will take

- The client must send the specified string to the server.
  Once received, the server must print it.
- The server must display the string without delay. If it seems slow, it is likely too slow.

`If displaying 100 characters takes 1 second, the program is too slow!`

- Your server should be able to receive strings from several clients in a row without needing to restart.
- Communication between the client and server must `exclusively` use UNIX signals.
- You can only use these two signals: `SIGUSR1` and `SIGUSR2`.

`Linux system does NOT queue signals when you already have pending signals of this type! Bonus time?`

Chapter VI

Readme Requirements

A `README.md` file must be provided at the root of your Git repository. Its purpose is to allow anyone unfamiliar with the project (peers, staff, recruiters, etc.) to quickly understand what the project is about, how to run it, and where to find more information on the topic.
The `README.md` must include at least:

- The very first line must be italicized and read: *This project has been created as part of the 42 curriculum by <login1>[, <login2>[, <login3>[...]]].*
- A "**Description**" section that clearly presents the project, including its goal and a brief overview.
- An "**Instructions**" section containing any relevant information about compilation, installation, and/or execution.
- A "**Resources**" section listing classic references related to the topic (documentation, articles, tutorials, etc.), as well as a description of how AI was used — specifying for which tasks and which parts of the project.

➠ **Additional sections may be required depending on the project** (e.g., usage examples, feature list, technical choices, etc.).

Any required additions will be explicitly listed below.

`English is recommended; alternatively, you may use the main language of your campus.`

Chapter VII

Bonus part

Bonus list:

- The server must acknowledge each received message by sending a signal to the client.
- Unicode characters support!

The bonus part will only be assessed if the mandatory part is PERFECT. Perfect means the mandatory part has been fully completed and functions without any errors. If you have not met ALL the mandatory requirements, the bonus part will not be evaluated.

Chapter VIII

Submission and peer-evaluation

Submit your assignment in your `Git` repository as usual. Only the content within your repository will be assessed during the defense. Don't hesitate to double-check the names of your files to ensure they are correct.

During the evaluation, a brief **modification of the project** may occasionally be requested. This could involve a minor behavior change, a few lines of code to write or rewrite, or an easy-to-add feature.

While this step may **not be applicable to every project**, you must be prepared for it if it is mentioned in the evaluation guidelines.

This step is meant to verify your actual understanding of a specific part of the project. The modification can be performed in any development environment you choose (e.g., your usual setup), and it should be feasible within a few minutes — unless a specific timeframe is defined as part of the evaluation.
You can, for example, be asked to make a small update to a function or script, modify a display, or adjust a data structure to store new information, etc.

The details (scope, target, etc.) will be specified in the **evaluation guidelines** and may vary from one evaluation to another for the same project.